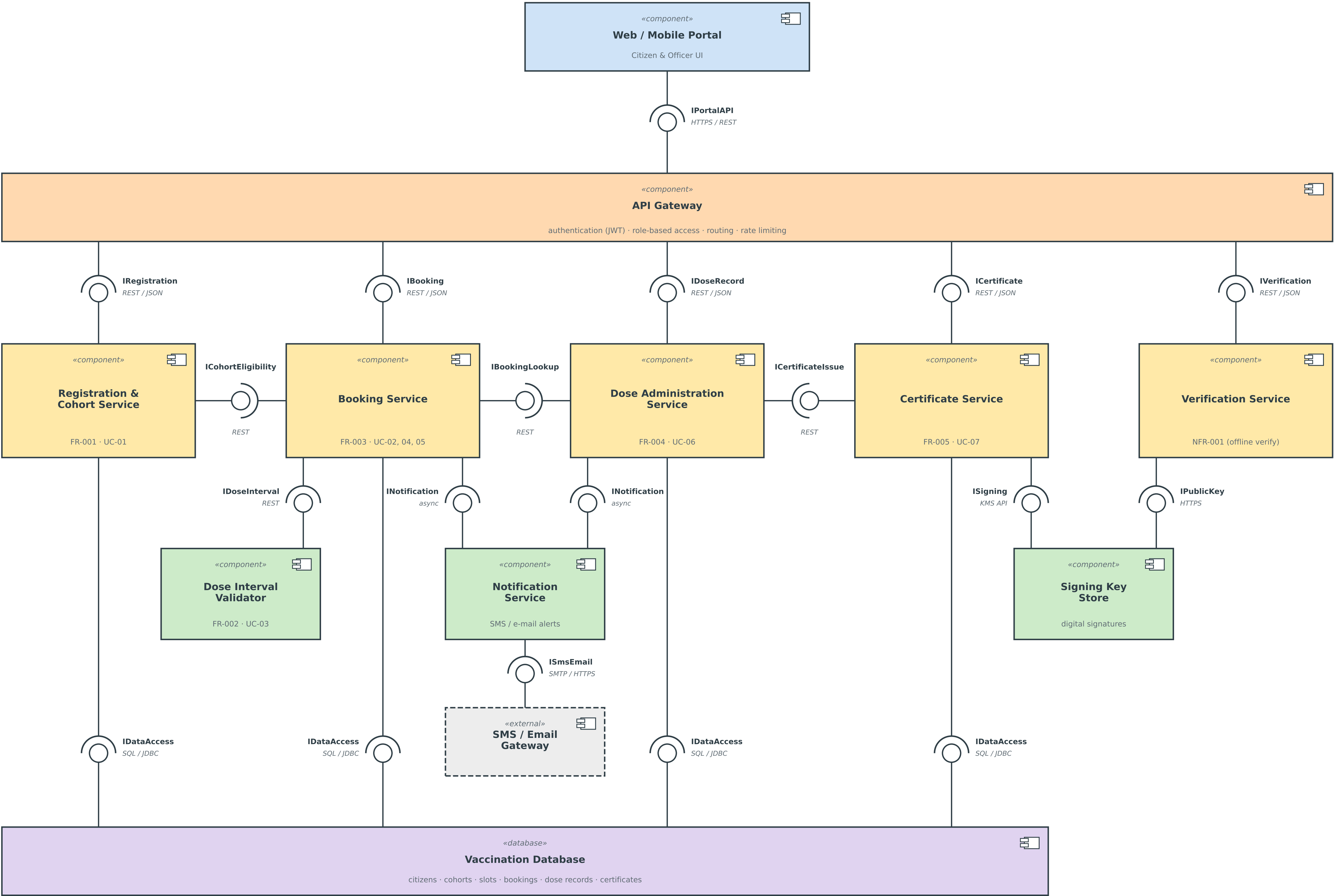


Vaccination Cohort & Dose Scheduling System — UML Component Diagram

Microservices Architecture | PES University · SE Lab 3 · Problem Statement #17



Legend



Client



Edge / entry point



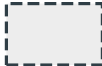
Business microservice



Supporting component



Data store



External system



Provided interface (ball)



Required interface (socket)

Ball + socket joined by a solid line = assembly connector (labelled: interface name / protocol)

## Lab 3: Architectural Justification

Vaccination Cohort & Dose Scheduling System | Problem Statement #17 | PES University, SE Lab 3

**Architecture Selection:** We chose **Microservices Architecture** for the **Vaccination Cohort & Dose Scheduling System**.

### Architectural Choice

The system is split into independently deployable services behind a single API Gateway: Registration & Cohort, Booking, Dose Administration, Certificate and Verification, supported by a Dose Interval Validator, Notification Service, Signing Key Store and a shared Vaccination Database. Services talk only through the interfaces shown in the component diagram (19 provided/required interface pairs).

### Two Reasons

**1. Vaccination drives create sharp, uneven peaks (NFR-002).** The system must hold 99.9% uptime and auto-scale once concurrent sessions pass 5,000. Slot browsing and booking (FR-003) take most of the load, while registration and dose recording take far less. Microservices let us scale only Booking and Registration when demand spikes. A failure in a peripheral service such as Notification does not stop a citizen from booking, which protects the uptime target.

**2. Different parts of the system have very different requirements (NFR-001, FR-002).** Certificate verification must answer in under 150 ms for 10,000 concurrent requests, including offline, so it is a separate, stateless Verification Service that never competes with booking traffic. The dose-interval rule (e.g. 28 days) is isolated in its own Validator component, so it can change when the medical protocol changes without redeploying Booking.

### Security Advantage

The API Gateway is the only entry point. It authenticates every request (JWT) and enforces roles, so a Citizen Registrant cannot call Vaccination Officer operations such as recording a dose (FR-004). Internal services are not exposed to the internet. The private signing key lives only in the Signing Key Store and is used only by the Certificate Service; the Verification Service receives the public key only. A compromise of one service therefore cannot forge certificates (FR-005), and sensitive identity data (Aadhaar/ID) stays confined to Registration and the database.

### Performance Benefit

Stateless services scale horizontally and independently, so extra Booking or Verification instances start within the 60-second target without touching the rest of the system. Notification calls are asynchronous, so SMS/e-mail delays stay off the booking response path. Verification checks signatures against a cached public key, which allows the sub-150 ms, offline-capable response required by NFR-001.

### Alternatives Considered

| Style         | Strength                                   | Why it was not chosen                                                                                                                        |
|---------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Layered       | Simple, clear separation of concerns       | The whole stack scales as one unit, so it cannot scale booking alone during peaks, and tight layer coupling makes rule changes risky.        |
| Client-Server | Centralised control, easy data consistency | A single server is a single point of failure and a bottleneck, which conflicts with the 99.9% uptime and 5,000-session auto-scaling targets. |

**Trade-off accepted:** microservices add operational complexity and network latency between services. We accept this because the availability, scaling and verification-speed requirements outweigh it.